PROGRAMACIÓN COMPETITIVA · ICPC

# Algoritmario ICPC

Referencia rápida de algoritmos para competencia

| 48 | 9 | 2,485 |
| --- | --- | --- |
| ALGORITMOS | CATEGORÍAS | LÍNEAS DE CÓDIGO |

EDICIÓN GENERADA EL 9 DE OCTUBRE DE 2026

# Índice

Las categorías y los algoritmos están enlazados. Haz clic en cualquier entrada para ir directamente a su página.

**Data-structures . . . 4**

bint . . . 5
bit_purq . . . 6
bit_rupq . . . 7
dsu . . . 8
monotonic_queue . . . 9
segment_tree . . . 10
sparse_table . . . 11

**Dynamic-programming . . . 12**

knapsack_01_optimization . . . 13
knuth_optimization . . . 14
lis . . . 15

**Geometry . . . 16**

circle_line_intersection . . . 17
closest_pair . . . 18
convex_hull . . . 19
formulas . . . 20
Geometry_template_2d . . . 22
Line_intersec . . . 24
point_inside_convex_polygon . . . 25
Point_Inside_Polygon . . . 26
Polygon_Area . . . 27

**Graph . . . 28**

bellman_ford . . . 29
binary_lifting . . . 30
cycle_detection_directed_graph . . . 31
dijkstra . . . 32
floyd_warshall . . . 33
mst . . . 34
tarjan . . . 35
tarjan_bridges_articulation . . . 37

tarjan_scc . . . 38

**Graph / Flows . . . 39**

dinic . . . 40

edmonds_karp . . . 45

min_cost_max_flow . . . 46

**Graph / Tree . . . 48**

diameter . . . 49

lowest_common_ancestor . . . 50

**Math . . . 51**

binary_exponentiation . . . 52

combinatorics . . . 53

fft . . . 54

matrix_expo . . . 57

modular_arithmetic . . . 58

precision . . . 59

prime_theory . . . 60

**Misc . . . 61**

jose . . . 62

random . . . 63

**String . . . 64**

aho_corasick . . . 65

hashing . . . 66

kmp . . . 67

suffix_array . . . 68

Trie . . . 73

z_funtion . . . 74

SECCIÓN 01

# Data-structures

7 algoritmos

# bint

DATA-STRUCTURES · C++ · 63 LÍNEAS

CÓDIGO · bint

```cpp
const int base {1'000'000'000};
class Bint {
public:
    Bint(int num) : d(1, num) {};
    Bint(std::string& num) {
        int n{std::ssize(num)};
        for (int i{n}; i > 0; i -= 9) {
            d.push_back(std::stoi((i >= 9) ? num.substr(i - 9, 9) : num.substr(0, i)));
        }
    }
    Bint operator+(Bint& o) {
        Bint bint{}; auto& ans{bint.d}; int carry{};
        for (int i{}; i < std::max(size(), o.size()); ++i) {
            auto x{(i < size()) ? d[i] : 0};
            auto y {(i < o.size()) ? o[i] : 0};
            auto v {(x + y + carry)};
            ans.push_back(v % base);
            carry = (v >= base);
        }
        if (carry) ans.push_back(carry);
        bint.trim();
        return bint;
    }
    Bint operator*(int b) {
        Bint bint{};
        auto& ans {bint.d};
        long long c{};
        for (int i{}; i < size(); ++i) {
            long long x {(d[i] * 1ll * b) + c};
            ans.push_back(x % base);
            c = (x / base);
        }
        while (c) ans.push_back(c % base); c /= base;
        bint.trim();
        return bint;
    }
    bool operator==(Bint& o) {
        if (size() != o.size()) return false;
        for (int i{}; i < size(); ++i) if (d[i] != o[i]) return false;
        return true;
    }
    bool operator<(Bint& o) {
        if (size() != o.size()) return size() < o.size();
        for (int i{size() - 1}; i >= 0; --i) if (d[i] != o[i]) return d[i] < o[i];
        return false;
    }
    std::string toString() {
        if (d.empty()) return "0";
        std::string s{std::format("{}", d.back())};
        s.reserve(9 * size());
        for (int i{size() - 2}; i >= 0; --i) {
            s += std::format("{:09d}", d[i]);
        }
        return s;
    }
private:
    std::vector<int> d{};
    int operator[](int i) { return d[i]; }
    int size() { return std::ssize(d); }
    void trim() {
        while (size() > 1 && d.back() == 0) d.pop_back();
    }
};
```

# bit_purq

**DATA-STRUCTURES · C++ · 48 LÍNEAS**

**CÓDIGO · bit_purq**

```cpp
class BitPURQ {
public:
    BitPURQ (const std::vector<long long>& a)
        : n{static_cast<int>(a.size()) + 1}, ft(n) {
            for (int i{1}; i < n; ++i) {
                ft[i] += a[i - 1];
                if (i + lsb(i) < n) ft [i + lsb(i)] += ft[i];
            }
        }

    long long query(int l, int r) {
        return sum(r) - sum(l - 1);
    }

    void increment(int i, long long delta) {
        while (i < n) {
            ft[i] += delta;
            i  += lsb(i);
        }
    }

    int find(long long k) {
        int i{}, b{1};
        while (b * 2 < n) b *= 2;

        for (int j{b}; j; j >>= 1) {
            if (i + j < n && ft[i + j] < k) {
                k -= ft[i + j];
                i += j;
            }
        }
        return i + 1;
    }
private:
    int n{};
    std::vector<long long> ft{};

    int lsb(int i) { return (i & (-i)); }

    long long sum(int i) {
        long long ans{};
        while (i) {
            ans += ft[i];
            i -= lsb(i);
        }
        return ans;
    }
};
```

# bit_rupq

**DATA-STRUCTURES · C++ · 35 LÍNEAS**

**CÓDIGO · bit_rupq**

```cpp
class BitRUPQ {
public:
    BitRUPQ (const std::vector<long long>& a)
        : n{static_cast<int>(a.size()) + 1}, ft(n) {
            for (int i{1}; i < n; ++i) {
                increment(i, i, a[i - 1]);
            }
        }

    long long value (int i) {
        long long ans{};
        while (i) {
            ans += ft[i];
            i -= lsb(i);
        }
        return ans;
    }

    void increment(int l, int r, long long delta) {
        sum(l, delta);
        sum(r + 1, -delta);
    }
private:
    int n{};
    std::vector<long long> ft{};

    int lsb(int i) { return (i & (-i)); }

    void sum(int i, long long delta) {
        while (i < n) {
            ft[i] += delta;
            i += lsb(i);
        }
    }
};
```

# dsu

DATA-STRUCTURES · C++ · 29 LÍNEAS

CÓDIGO · dsu

```cpp
class UF {
public:
    UF (int n)
        : c{n}, p(n), sz(n, 1) {
            std::iota(p.begin(), p.end(), 0);
        }

    int find (int i) {
        while (p[i] != i) i = p[i] = p[p[i]];
        return i;
    }

    bool unite(int i, int j) {
        i = find(i); j = find(j);
        if (i == j) return false;
        if (sz[i] < sz[j]) std::swap(i, j);
        p[j] = i;
        sz[i] += sz[j];
        --c;
        return true;
    }

    int operator[](int i) { return sz[find(i)]; }
    int components() { return c; }
    bool same (int i, int j) { return find(i) == find(j); }
private:
    int c{};
    std::vector<int> p{}, sz{};
};
```

# monotonic_queue

**DATA-STRUCTURES · C++ · 23 LÍNEAS**

**# CÓDIGO · monotonic_queue**

```cpp
// Monotonic Stack
std::vector<int> stk{-1};
stk.reserve(n + 1);

for (int i{}; i < n; ++i) {
    while (stk.back() != -1 && a[stk.back()] <= a[i]) stk.pop_back();
    // process
    stk.push_back(i);
}

// Monotonic deque
int m { /* window size */};
std::deque<int> dq{};

for (int i{}; i < n; ++i) {
    while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
    dq.push_back(i);
    if (dq.front() <= i - m) dq.pop_front();

    if (i >= m - 1) {
        int max_value {a[dq.front()]};
    }
}
```

# segment_tree

**DATA-STRUCTURES · C++ · 59 LÍNEAS**

**CÓDIGO · segment_tree**

```cpp
class SegmentTree {
    using T = long long;
public:
    SegmentTree (const std::vector<T>& a)
        : n{static_cast<int>(a.size())}, tree(n * 4, kdefault), lazy(n * 4, klazy) {
            build(0, 0, n - 1, a);
        }
    void increment (int l, int r, T delta);
    T query        (int l, int r);
private:
    int n{};
    const T kdefault{}, klazy{};
    std::vector<T> tree{}, lazy{};

    T agg (T a, T b) { return std::max(a, b); }
    int left  (int node)     { return (node << 1) + 1; }
    int right (int node)     { return (node << 1) + 2; }
    int mid   (int a, int b) { return (a + b) / 2; }

    void build(int node, int l, int r, const std::vector<T>& a) {
        if (l == r) {
            tree[node] = a[l];
            return;
        }
        build(left(node), l, mid(l, r), a);
        build(right(node), mid(l , r) + 1, r, a);
        tree[node] = agg(tree[left(node)], tree[right(node)]);
    }
    void increment(int node, int l, int r, int leftq, int rightq, T delta) {
        propagate(node, l, r);
        if (r < leftq || rightq < l) return;
        if (leftq <= l && r <= rightq) {
            lazy[node] += delta;
            propagate(node, l, r);
            return;
        }
        increment(left(node), l, mid(l, r),      leftq, rightq, delta);
        increment(right(node), mid(l, r) + 1, r, leftq, rightq, delta);
        tree[node] = agg(tree[left(node)], tree[right(node)]);
    }
    T query(int node, int l, int r, int leftq, int rightq) {
        if (r < leftq || rightq < l) return kdefault;
        propagate(node, l, r);
        if (leftq <= l && r <= rightq) return tree[node];
        return agg (
            query(left(node), l, mid(l, r),      leftq, rightq),
            query(right(node), mid(l, r) + 1, r, leftq, rightq)
        );
    }
    void propagate (int node, int l, int r) {
        if (lazy[node] == klazy) return;
        tree[node] += lazy[node];
        if (l != r) {
            lazy[left(node)]  += lazy[node];
            lazy[right(node)] += lazy[node];
        }
        lazy[node] = klazy;
    }
};
```

# sparse_table

**DATA-STRUCTURES · C++ · 26 LÍNEAS**

**# CÓDIGO · sparse_table**

```cpp
class SparseTable {
public:
    SparseTable(const std::vector<int>& b)
        : n {static_cast<int>(b.size())}, lg(n + 1) {
        for (int i{2}; i <= n; ++i) lg[i] = lg[i / 2] + 1;
        m = lg[n] + 1;

        mat.resize(m * n);
        for (int i{}; i < n; ++i) v(i, 0) = b[i];

        for (int j{1}; j < m; ++j) {
            for (int i{}; i + (1 << j) - 1 < n; ++i) {
                v(i, j) = std::min(v(i, j - 1), v(i + (1 << (j - 1)), j - 1));
            }
        }
    }
    int query(int l, int r) {
        --l; --r;
        int k {lg[r - l + 1]};
        return std::min(v(l, k), v(r - (1 << k) + 1, k));
    }
private:
    int n{}, m{};
    std::vector<int> mat{}, lg{};
    int& v(int i, int j) { return mat[(j * n) + i]; }
};
```

SECCIÓN 02

# Dynamic-programming

3 algoritmos

# knapsack_01_optimization

**DYNAMIC-PROGRAMMING · C++ · 24 LÍNEAS**

**# CÓDIGO · knapsack_01_optimization**

```cpp
/*  0/1 Knapsack optimization where sum of all items is ~ N
    There will be at most sqrt(N) different items.
    Array 'cnt' represents the count of a specific item.
*/

const long long maxn = 1e5+50;
const long long maxnsq = 400;

long long n,m,cnt[maxn],dp[maxnsq][maxn];
vector<long long> c;
void calculateDp(){ //DP in O(N*sqrt(N))
    dp[0][0]=0;
    for(int i = 1; i < n+5; i++) dp[0][i] = -1;
    for(int i = 1; i < (int)c.size(); i++){ // c is the array of unique items.
        for(int j = 1; j < n+1; j++){
            if(dp[i-1][j] >= 0)
                dp[i][j] = 0;
            else if(j-c[i] >= 0 && dp[i][j - c[i]] >= 0 and dp[i][j - c[i]] < cnt[c[i]])
                dp[i][j] = dp[i][j - c[i]] + 1;
            else
                dp[i][j] = -1;
        }
    }
}
```

# knuth_optimization

**DYNAMIC-PROGRAMMING · C++ · 45 LÍNEAS**

**# CÓDIGO · knuth_optimization**

```cpp
/*
Optimization:
Special case of DP on ranges, that optimizes the time
complexity of solutions by a linear factor, from O(n^3)
to O(n^2).

Recurrence:
dp(i,j) = fore(k,i,j) min[dp(i,k) + dp(k+1,j) + C(i,j)]

We can apply this optimization when the cost function C
satisfies the following conditions for a <= b <= c <= d:
    1. C(b,c) <= C(a,d)
    2. C(a,c) + C(b,d) <= C(a,d) + C(b,c) (quad. ineq.)
*/

int solve() {
    int N;
    ... // read N and input
    int dp[N][N], opt[N][N];

    auto C = [&](int i, int j) {
        ... // Implement cost function C.
    };

    for (int i = 0; i < N; i++) {
        opt[i][i] = i;
        ... // Initialize dp[i][i] according to the problem
    }

    for (int i = N-2; i >= 0; i--) {
        for (int j = i+1; j < N; j++) {
            int mn = INT_MAX;
            int cost = C(i, j);
            for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k++) {
                if (mn >= dp[i][k] + dp[k+1][j] + cost) {
                    opt[i][j] = k;
                    mn = dp[i][k] + dp[k+1][j] + cost;
                }
            }
            dp[i][j] = mn;
        }
    }

    return dp[0][N-1];
}
```

# lis

**DYNAMIC-PROGRAMMING · C++ · 16 LÍNEAS**

**# CÓDIGO · lis**

```cpp
// Longest increasing subsequence O(nlogn)
const long long INF = 1e18;
int lis(const vector<long long> &a) {
    int n = (int)a.size();
    vector<long long> d(n+1, INF);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        int j = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
        if (d[j-1] < a[i] && a[i] < d[j]) d[j] = a[i];
    }

    int ans = 0;
    for(int i = 0; i < n+1; i++) if (d[i]<INF) ans = i;
    return ans;
}
```

SECCIÓN 03

# Geometry

9 algoritmos

# circle_line_intersection

**GEOMETRY · C++ · 19 LÍNEAS**

**CÓDIGO · circle_line_intersection**

```cpp
vector <Point> circleLineIntersection(double a, double b, double c, double r){
    //Dados los coeficientes de la ecuacion de la recta y el radio del circulo con centro en el origen
    vector <Point> pts;
    double x0 = -a*c/(a*a+b*b), y0 = -b*c/(a*a+b*b);
    if (c*c > r*r*(a*a+b*b)+EPS){} // 0 points.
    else if (abs (c*c - r*r*(a*a+b*b)) < EPS) pts.pb({x0,y0}); // 1 point.
    else{ // 2 points.
        double d = r*r - c*c/(a*a+b*b);
        double mult = sqrt (d / (a*a+b*b));
        double ax, ay, bx, by;
        ax = x0 + b * mult;
        bx = x0 - b * mult;
        ay = y0 - a * mult;
        by = y0 + a * mult;
        pts.pb({ax,ay});
        pts.pb({bx,by});
    }
    return pts;
}
```

# closest_pair

**GEOMETRY · C++ · 31 LÍNEAS**

**CÓDIGO · closest_pair**

```cpp
#define pii pair<ll,ll>

ll closestPair(vector <pii> pts){
    //Calcula el par de puntos en 2D mas cercanos entre si, retorna su distancia euclidiana.
    int n = sz(pts);
    sort(ALL(pts));
    set<pii> s;

    ll ans = INF;
    int pos = 0;
    fore(i,0,n){
        ll d = ceil(sqrt(ans));
        while (pts[i].first - pts[pos].first >= d) {
            s.erase({pts[pos].second, pts[pos].first});
            pos++;
        }

        auto it1 = s.lower_bound({pts[i].second - d, pts[i].first});
        auto it2 = s.upper_bound({pts[i].second + d, pts[i].first});

        for (auto it = it1; it != it2; it++) {
            ll dx = pts[i].first - it->second;
            ll dy = pts[i].second - it->first;
            if (ans > 1LL * dx * dx + 1LL * dy * dy){
                ans = 1LL * dx * dx + 1LL * dy * dy;
            }
        }
        s.insert({pts[i].second, pts[i].first});
    }
    return ans;
}
```

# convex_hull

**GEOMETRY · C++ · 23 LÍNEAS**

**# CÓDIGO · convex_hull**

```cpp
vector <Point> calculateHull(vector <Point> &p, int n){ //Calculo del Convex Hull
    if (n <= 2) return p;
    vector<Point> hull;
    int tam = 0;
    sort(p.begin(),p.end());
    fore(t,0,2){
        fore(i,0,n){
            while((int)hull.size()-tam >= 2){
                Point p1 = hull[(int)hull.size()-2];
                Point p2 = hull[(int)hull.size()-1];
                //Producto cruz: P1 es el ancla
                //agregar (<=) si tambien se quieren incluir los puntos colineales, sino solo (<)
                if(p1.cross(p2, p[i]) <= 0) break;
                hull.pop_back();
            }
            hull.push_back(p[i]);
        }
        hull.pop_back();
        tam = (int)hull.size();
        reverse(p.begin(),p.end());
    }
    return hull;
}
```

# formulas

**GEOMETRY · C++ · 79 LÍNEAS**

CÓDIGO · formulas

```cpp
// Volume of a sphere.
ld volumeSphere(ld rad){
    return (4.0/3.0)*PI*rad*rad*rad;
}

// Volume of a sphere cap.
ld volumeCap(ld h, ld rad){
    return PI*h*h*(rad-(h/3.0));
}

// Area of a triangle given vertices A, B, and C
ld areaTriangle(Point A, Point B, Point C) {
    return fabs((A.x * (B.y - C.y) + B.x * (C.y - A.y) + C.x * (A.y - B.y)) / 2.0);
}

// Area of a circle
ld areaCircle(ld radius) {
    return PI * radius * radius;
}

// Area of a trapezoid given bases and height
ld areaTrapezoid(ld base1, ld base2, ld height) {
    return 0.5 * (base1 + base2) * height;
}

// Volume of a cone
ld volumeCone(ld radius, ld height) {
    return (PI * radius * radius * height) / 3.0;
}

// Volume of a cylinder
ld volumeCylinder(ld radius, ld height) {
    return PI * radius * radius * height;
}

// Volume of a rectangular prism
ld volumeRectPrism(ld length, ld width, ld height) {
    return length * width * height;
}

// Volume of a pyramid with a rectangular base
ld volumePyramid(ld length, ld width, ld height) {
    return (length * width * height) / 3.0;
}

// Area of a parallelogram given two vectors (base and height)
ld areaParallelogram(Point base, Point heightVec) {
    return fabs(base * heightVec);
}

// Perimeter of a polygon given vertices (assuming vertices are in order)
ld perimeterPolygon(vector<Point> &vertices) {
    ld perimeter = 0.0;
    fore(i, 0, sz(vertices)) {
        perimeter += vertices[i].dist(vertices[(i + 1) % sz(vertices)]);
    }
    return perimeter;
}

// Volume of a prism with base area and height
ld volumePrism(ld baseArea, ld height) {
    return baseArea * height;
}

// Surface area of a sphere
ld surfaceAreaSphere(ld radius) {
```

**CÓDIGO · formulas**

```
    return 4 * PI * radius * radius;
}

// Surface area of the cap of a sphare
// Note: the radius is the radius of the whole sphere.
ld capSurfaceAreaSphere(ld height, ld radius){
    return 2 * PI * height * radius;
}

// Surface area of a cylinder
ld surfaceAreaCylinder(ld radius, ld height) {
    return 2 * PI * radius * (radius + height);
}
```

# Geometry_template_2d

**GEOMETRY · C++ · 72 LÍNEAS**

CÓDIGO · Geometry_template_2d

```cpp
#include <bits/stdc++.h>
#define ll long long
#define ld long double
#define PI acos(-1.0)

using namespace std;

struct Point {
    ld x, y;
    Point operator +(const Point& b) const { return Point{x+b.x, y+b.y};}
    Point operator -(const Point& b) const { return Point{x-b.x, y-b.y}; }
    ll operator *(const Point& b) const { return (ll) x * b.y - (ll) y * b.x; }
    Point operator *(const ld k)const{return Point{x*k,y*k};}
    bool operator <(const Point& b) const { return x == b.x ? y < b.y : x < b.x; }
    void operator +=(const Point& b) { x += b.x; y += b.y; }
    void operator -=(const Point &b) { x -= b.x; y -= b.y; }
    void operator *=(const int k) { x *= k; y *= k; }
    bool operator ==(const Point &b){
        if (b.x == (*this).x && b.y == (*this).y) return true;
        return false;
    }
    ld magnitude() const { return sqrt((x*x) + (y*y)); }
    ld dot (const Point &b){ return (x * b.x) + (y * b.y); }
    // If dot product > 0, angle between vectors is acute, = 0 vectors are perpendicular, and if < 0, angle
        is obtuse
    ld dist (const Point & b){return (*this - b).magnitude();}
    ll cross(const Point& b, const Point& c) const {
        ll cruz = (b - *this) * (c - *this);
        if (cruz < 0) return -1; //cw (right)
        if (cruz > 0) return +1; //ccw (left)
        return 0; //Collinear.
    }
    ld rawCross(const Point &a, const Point &b) const {
        return (a - *this) * (b - *this);
    }
    bool onSegment(Point p, Point r){ //point in a rectangular box
        if ((*this).x <= max(p.x, r.x) && (*this).x >= min(p.x, r.x) && (*this).y <= max(p.y, r.y) && (*
            this).y >= min(p.y, r.y)) return true;
        return false;
    }
    ld angleBetweenVectors(const Point &b){ //this: (b-a), Point b: (c-a).
        ld ang = acos((*this).dot(b)/((*this).magnitude() * b.magnitude()));
        ang = (ang * 180.0) / PI;
        return ang; //return angle in degrees.
    }
    int half() const { // for angular sorting, to know in which half is this
        if ((*this).y > 0) return 0;
        if ((*this).y < 0) return 1;
        return (*this).x >= 0 ? 0 : 1;
    }
};
struct LineToPoint{
    Point p1,p2;
    ld dist(Point refPoint){
        return abs((refPoint - p1) * (refPoint - p2)) / p1.dist(p2);
    }
};
ld degreesToRadians(ld degrees) { return degrees * PI / 180.0; }
ld radiansToDegrees(ld radians){ return radians * (180.0 / PI); }
Point rotate45(Point p){ // Rotates a point 45 degrees
    return {p.x + p.y, p.y - p.x};
}
Point undorotate45(Point p){ // Undo 45 degrees rotation of a point
    return {(p.x-p.y)/2, (p.y+p.x)/2};
}
bool angleCmp(const Point& a, const Point& b) { // for angular sorting
    int ha = a.half();
```

# CÓDIGO · Geometry_template_2d

```
    int hb = b.half();
    if (ha != hb) return ha < hb;              // upper half first
    ll cr = a*b;
    if (cr != 0) return cr > 0;                // CCW order
    // Collinear: tie-break by distance from origin (shorter first, optional)
    return a.magnitude() < b.magnitude();
}
```

# Line_intersec

**GEOMETRY · C++ · 14 LÍNEAS**

**# CÓDIGO · Line_intersec**

```cpp
bool doIntersect(Point p1, Point q1, Point p2, Point q2){ //Checa si 2 lineas se intersectan o no.
    int o1 = p1.cross(q1, p2);
    int o2 = p1.cross(q1, q2);
    int o3 = p2.cross(q2, p1);
    int o4 = p2.cross(q2, q1);

    if (o1 != o2 && o3 != o4) return true;
    if (o1 == 0 && p2.onSegment(p1, q1)) return true;
    if (o2 == 0 && q2.onSegment(p1, q1)) return true;
    if (o3 == 0 && p1.onSegment(p2, q2)) return true;
    if (o4 == 0 && q1.onSegment(p2, q2)) return true;

    return false;
}
```

# point_inside_convex_polygon

**GEOMETRY · C++ · 61 LÍNEAS**

**# CÓDIGO · point_inside_convex_polygon**

```cpp
//Este codigo asume orden antihorario

int sgn(ll val) { return val > 0 ? 1 : (val == 0 ? 0 : -1); }

bool pointInTriangle(Point a, Point b, Point c, Point point){
    ll s1 = abs(a.rawCross(b, c));
    ll s2 = abs(point.rawCross(a, b)) + abs(point.rawCross(b, c)) + abs(point.rawCross(c, a));
    return s1 == s2;
}

//Precalculation for queries to know if a point lies inside of a convex polygon.
void prepareConvexPolygon(int &n, vector<Point> &points, vector<Point> &seq, Point &translation){ //seq and
    translation are empty here.
    n = points.size();
    int pos = 0;
    for (int i = 1; i < n; i++) {
        if (points[i] < points[pos])
            pos = i;
    }
    rotate(points.begin(), points.begin() + pos, points.end());

    n--;
    seq.resize(n);
    for (int i = 0; i < n; i++)
        seq[i] = points[i + 1] - points[0];
    translation = points[0];
}

//Know if a point lies inside of a convex polygon in O(logN)
bool pointInConvexPolygon(Point point, int &n, vector<Point> &seq, Point &translation) {
    point = point - translation;
    if (seq[0]*point != 0 && sgn(seq[0]*point) != sgn(seq[0]*seq[n-1])) return false;
    if (seq[n - 1]*point != 0 && sgn(seq[n - 1]*point) != sgn(seq[n - 1]*seq[0])) return false;
    if (seq[0]*point == 0) return seq[0].dot(seq[0]) >= point.dot(point);
    /*
    Si no acepta puntos en el borde remplaza linea anteriror con
    if (seq[0] * point == 0) return false;
    if (seq[n - 1] * point == 0) return false;
    */
    int l = 0, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) / 2;
        int pos = mid;
        if (seq[pos]*point >= 0) l = mid;
        else r = mid;
    }
    int pos = l;
    return pointInTriangle(seq[pos], seq[pos + 1], Point{0,0}, point);
    /*
    Mismo caso que no acepte cosas en el borde (Strictly inside)
    return (seq[pos + 1] - seq[pos]) * (point - seq[pos]) > 0;
    */
}

void doit(){
    int n;
    vector <Point> poly; //with input.
    vector <Point> seq; //empty.
    Point translation;
    prepareConvexPolygon(n,poly,seq,translation);
    // then call pointInConvexPolygon() for queries.
}
```

# Point_Inside_Polygon

**GEOMETRY · C++ · 18 LÍNEAS**

**CÓDIGO · Point_Inside_Polygon**

```cpp
// Checa si un punto dado esta DENTRO, FUERA o en FRONTERA con un poligono
// Recuerda cerrar el poligono
string checkPointInsidePolygon(vector <Point> P, Point point, int n){
    P[0] = point;
    ll count = 0;
    if (n < 3) return "OUTSIDE";
    for(int i = 1; i <= n; i++){
        int j = (i == n ? 1 : i+1);
        if(P[i].x <= P[0].x && P[0].x < P[j].x && P[0].cross(P[i], P[j]) < 0) count++;
        else if(P[j].x <= P[0].x && P[0].x < P[i].x && P[0].cross(P[j], P[i]) < 0)  count++;

        if ((min(P[i].x,P[j].x) <= point.x && point.x <= max(P[i].x,P[j].x)) && (min(P[i].y,P[j].y) <=
            point.y && point.y <= max(P[i].y,P[j].y)) && point.cross(P[i],P[j]) == 0){
            return "BOUNDARY";
        }
    }
    if (count%2 == 1) return "INSIDE";
    return "OUTSIDE";
}
```

# Polygon_Area

**GEOMETRY · C++ · 6 LÍNEAS**

**CÓDIGO · Polygon_Area**

```
ld getPolygonArea(vector <Point> &poly){ //Calculo de area de poligono
    ll ans = 0;
    poly.push_back(poly.front());
    for(int i = 1; i<(int)poly.size(); i++) ans += (poly[i-1]*poly[i]);
    return abs(ans);
}
```

SECCIÓN 04

# Graph

9 algoritmos

# bellman_ford

**GRAPH · C++ · 27 LÍNEAS**

**CÓDIGO · bellman_ford**

```cpp
dis[start] = 0;
for (int i{}; i < n - 1; ++i) {
    for (int node{}; node < n; ++node) {
        if (dis[node] == INF) continue;
        for (auto [u, w] : adj[node]) {
            dis[u] = std::min(dis[u], dis[node] + w);
        }
    }
}
std::vector<int> ciclosos(n), q{};
for (int node{}; node < n; ++node) {
    if (dis[node] == INF) continue;
    for (auto [u, w] : adj[node]) {
        if (dis[node] + w < dis[u]) {
            ciclosos[u] = true;
            q.push_back(u);
        }
    }
}
for (int j{}; j < std::ssize(q); ++j) {
    auto node {q[i]};
    for (auto [u, w] : adj[node]) {
        if (ciclosos[u]) continue;
        ciclosos[u] = true;
        q.push_back(u);
    }
}
```

# binary_lifting

**GRAPH · C++ · 30 LÍNEAS**

**# CÓDIGO · binary_lifting**

```cpp
#define ll long long
#define fore(i,a,b) for(int i = a; i<b; i++)
#define rofe(i,a,b) for(ll i=a-1;i>=b;--i)

const ll maxn = 2e5+5;
const ll loga = 20;
ll n,up[maxn][loga],dep[maxn];

void binlift(){
    // Assuming we have all direct parents for each node (up[node][0])
    fore(i,1,loga){
        fore(j,1,n+1) up[j][i] = up[up[j][i-1]][i-1];
    }
}

ll lca(ll x, ll y){ //calculate the depths of each node before.
    if (dep[x] < dep[y]) swap(x,y);
    ll dif = dep[x]-dep[y];
    rofe(i,loga,0){
        if (dif & (1ll<<i)) x = up[x][i];
    }
    if (x == y) return x;
    rofe(i,loga,0){
        if (up[x][i] != up[y][i]){
            x = up[x][i];
            y = up[y][i];
        }
    }
    return up[x][0];
}
```

# cycle_detection_directed_graph

**GRAPH · C++ · 33 LÍNEAS**

**CÓDIGO · cycle_detection_directed_graph**

```cpp
enum Colors {none, explored, visited};
std::vector<int> vis(n);
auto hasCycle = [&](auto& self, int node) -> bool {
    vis[node] = explored;
    for (auto u : adj[node]) {
        if (vis[u] == none) {
            if (self(self, u)) return true;
        } else if (vis[u] == explored) return true;
    }
    vis[node] = visited;
    return false;
};

// kanh algorithm
std::vector<int> q{};
q.reserve(n);
for (int i{}; i < n; ++i) {
    if (inDegree[i] == 0) {
        q.push_back(i);
    }
}
for (int j{}; j < std::ssize(q); ++j) {
    auto node {q[j]};
    for (auto u : adj[node]) {
        --inDegree[u];
        if (inDegree[u] == 0) {
            q.emplace_back(u);
        }
    }
}
if (std::ssize(q) != n) {
    std::cout << "There is a cycle.\n";
}
```

# dijkstra

**GRAPH · C++ · 23 LÍNEAS**

**# CÓDIGO · dijkstra**

```cpp
std::vector<int> dis(n, INF);

using T = std::pair<long long, int>;
std::priority_queue<T, std::vector<T>, std::greater<T>> pq{};

dis[start] = 0;
pq.emplace(dis[start], start);

std::vector< std::vector< std::pair<int,int>>> path(n);

while (!pq.empty()) {
    auto [cost, node] {pq.top()}; pq.pop();
    if (cost > dis[node]) continue;
    for (auto [u, w] : adj[node]) {
        if (dis[node] + w < dis[u]) {
            dis[u] = dis[node] + w;
            pq.emplace(dis[u], u);
            path[u] = {std::pair{node, w}};
        } else if (dis[node] + w == dis[u]) {
            path[u].emplace_back(node, w);
        }
    }
}
```

# floyd_warshall

**GRAPH · C++ · 39 LÍNEAS**

**# CÓDIGO · floyd_warshall**

```cpp
const int nax {500};
std::array< std::array<int, nax>, nax> dis{}, p{};

const int INF {1'000'000'000};
for (auto& row : dis) row.fill(INF);
for (auto& row : p) row.fill(-1);

for (int b{}; b < n; ++b) {
    for (int i{}; i < n; ++i) {
        for (int j{}; j < n; ++j) {
            if (dis[i][b] == INF || dis[b][j] == INF) continue;
            if (dis[i][j] > dis[i][b] + dis[b][j]) {
                dis[i][j] = dis[i][b] + dis[b][j];
                p[i][j] = b;
            }
        }
    }
}

int cycleSize{INF};
for (int i{}; i < n; ++i) {
    cycleSize = std::min(cycleSize, dis[i][i]);
}

std::vector<int> ans{};
for (int I{}; I < n; ++I) {
    if (dis[I][I] == cycleSize){
        auto reconstruct = [&](auto& self, int i, int j) -> void {
            if (p[i][j] == -1) {
                ans.push_back(i);
                return;
            }
            auto k {p[i][j]};
            self(self, i, k);
            self(self, k, j);
        };
        reconstruct(reconstruct, I, I);
    }
}
```

# mst

**GRAPH · C++ · 33 LÍNEAS**

**CÓDIGO · mst**

```cpp
// Kruskal
std::vector<std::tuple<int,int,int>> edges{};
std::ranges::sort(edges);

UF dsu{n};
std::vector<std::vector<int>> adj(n);
for (auto [weight, u, v] : edges) {
    if (dsu.unite(u, v)) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    if (dsu.components() == 1) break;
}

// PRIM
using T = std::pair<int, int>;
std::priority_queue<T, std::vector<T>, std::greater<T>> pq{};
std::vector<int> vis(n);
auto process = [&](int node) {
    vis[node] = true;
    for (auto [u, w] : adj[node]) {
        if (vis[u]) continue;
        pq.emplace(w, u);
    }
};
process(0);
long long cost{};
while (!pq.empty()) {
    auto [w, node] {pq.top()}; pq.pop();
    if (vis[node]) continue;
    cost += w;
    process(node);
}
```

# tarjan

**GRAPH · C++ · 73 LÍNEAS**

**CÓDIGO · tarjan**

```cpp
const long long maxn = 2e5+10;
long long n,x,y,m,timer=0;
long long low[maxn],disc[maxn],isArt[maxn],inStack[maxn];
vector<long long> adj[maxn];
vector<vector<long long>> scc;
vector < pair<long long, long long> > brid;
stack <int> stk; // to find SCCs (directed graphs)
vector <pair<long long, long long>> edges; // to find BCCs (undirected graphs)

void extract_bcc_nodes(int stop_u, int stop_v, vector<long long> &bcc_nodes){
    while((int)edges.size()){
            auto e = edges.back();
            edges.pop_back();
            bcc_nodes.push_back(e.first);
            bcc_nodes.push_back(e.second);
            if ((e.first == stop_u && e.second == stop_v) || (e.first == stop_v && e.second == stop_u))
                break;
        }
        sort(bcc_nodes.begin(), bcc_nodes.end());
        bcc_nodes.erase(unique(bcc_nodes.begin(), bcc_nodes.end()), bcc_nodes.end());
}

void dfs(int node, int antnode = -1){
    low[node] = disc[node] = ++timer;
    stk.push(node);
    inStack[node] = 1;
    int children = 0;
    for (auto signode : adj[node]){
        if(disc[signode] == 0){
            children++;
            edges.push_back({node,signode});
            dfs(signode, node);

            // To find bridges:
            if (low[signode] > disc[node]){
                brid.push_back({node,signode});
            }

            low[node] = min(low[node], low[signode]);

            // To find articulation points (undirected graphs):
            if (antnode == -1 && children > 1) isArt[node] = 1;
            if (antnode != -1 && low[signode] >= disc[node]) isArt[node] = 1;

            // To find BCCs (undirected graphs):
            if (low[signode] >= disc[node]){
                vector<long long> bcc_nodes; // stores the nodes from a BCC (in no specific order)
                extract_bcc_nodes(node,signode,bcc_nodes);
            }
        }
        // To find SCC's (directed graphs):
        else if (inStack[signode] == 1){
            low[node] = min(low[node], disc[signode]);
        }
        // To find articulation points, bridges, and/or BCCs (undirected graphs)
        else if (antnode != signode){
            if (disc[signode] < disc[node]){ // to find BCCs
                edges.push_back({node,signode});
            }
            low[node] = min(low[node], disc[signode]);
        }
    }
    if (low[node] == disc[node]){ // To find SCC's (directed graphs)
        vector<long long> scctem;
        while (true){
            long long topic = stk.top();
            stk.pop();
```

**# CÓDIGO · tarjan**

```
            scctem.push_back(topic);
            inStack[topic] = 0;
            if (node == topic) break;
        }
        scc.push_back(scctem);
    }
}
```

# tarjan_bridges_articulation

**GRAPH · C++ · 32 LÍNEAS**

**CÓDIGO · tarjan_bridges_articulation**

```cpp
// Undirected edges {vertex, id}
std::vector<std::vector<std::pair<int,int>>> adj(n);

std::vector<int> bridges(m), articulation(n);
int root{}, childrenRoot{};

std::vector<int> pre(n), low(n);
int cont{};
auto ldfs = [&](auto& dfs, int node, int parentEdge) -> void {
    pre[node] = low[node] = ++cont;
    for (auto [u, id] : adj[node]) {
        if (id == parentEdge) continue;
        if (pre[u] == 0) {
            // forward / tree edge
            dfs(dfs, u, id);
            if (node == root) ++childrenRoot;
            if (low[u] > pre[node]) bridges[id] = true;
            if (low[u] >= pre[node]) articulation[node] = true;
            low[node] = std::min(low[u], low[node]);
        } else {
            // back edge
            low[node] = std::min(low[node], pre[u]);
        }
    }
};

for (int i{}; i < n; ++i) {
    if (pre[i]) continue;
    root = i; childrenRoot = 0;
    dfs(dfs, i, -1);
    articulation[i] = (childrenRoot > 1);
}
```

# tarjan_scc

**GRAPH · C++ · 32 LÍNEAS**

**CÓDIGO · tarjan_scc**

```cpp
const int INF {1'000'000'000};
std::vector<int> low(n), pre(n), id(n, -1);
std::stack<int> stk{};
stk.reserve(n);
int cont{}, scc{};

auto ldfs = [&](auto& dfs, int node) -> void {
    pre[node] = low[node] = ++cont;
    stk.push(node);

    for (auto u : adj[node]) {
        if (pre[u] == 0) {
            dfs(dfs, u);
        }
        low[node] = std::min(low[node], low[u]);
    }
    if (low[node] == pre[node]) {
        while (true) {
            auto x{stk.top()}; stk.pop();
            id[x] = scc;
            low[x] = INF;
            if (x == node) break;
        }
        ++scc;
    }
};

for (int i{}; i < n; ++i) {
    if (pre[i] == 0) {
        dfs(dfs, i);
    }
}
```

SECCIÓN 05

# Graph / Flows

3 algoritmos

# dinic

GRAPH / FLOWS · C++ · 341 LÍNEAS

CÓDIGO · dinic

```cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

struct FlowEdge {
    int from;
    int to;
    ll cap;
    ll flow;

    FlowEdge(int from, int to, ll cap)
        : from(from), to(to), cap(cap), flow(0) {}
};

struct Dinic {
    const ll INF = (ll)4e18;

    int n;
    int source;
    int sink;

    vector<FlowEdge> edges;
    vector<vector<int>> adj;

    vector<int> level;
    vector<int> ptr;

    Dinic(int n, int source, int sink)
        : n(n), source(source), sink(sink) {

        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    // Directed edge: from -> to with capacity cap
    void add_edge(int from, int to, ll cap) {
        int id = (int)edges.size();

        // Original edge
        edges.emplace_back(from, to, cap);

        // Residual edge
        edges.emplace_back(to, from, 0);

        adj[from].push_back(id);
        adj[to].push_back(id + 1);
    }

    // Builds the level graph
    bool bfs() {
        fill(level.begin(), level.end(), -1);

        queue<int> q;

        level[source] = 0;
        q.push(source);

        while (!q.empty()) {
            int v = q.front();
            q.pop();

            for (int id : adj[v]) {
                FlowEdge& e = edges[id];
```

# CÓDIGO · dinic

```
                // No residual capacity
                if (e.cap - e.flow <= 0) {
                    continue;
                }

                // Already visited
                if (level[e.to] != -1) {
                    continue;
                }

                level[e.to] = level[v] + 1;
                q.push(e.to);
            }
        }

        return level[sink] != -1;
    }

    // Sends flow through the level graph
    ll dfs(int v, ll pushed) {
        if (pushed == 0) {
            return 0;
        }

        if (v == sink) {
            return pushed;
        }

        for (int& cid = ptr[v];
             cid < (int)adj[v].size();
             ++cid) {

            int id = adj[v][cid];
            FlowEdge& e = edges[id];

            if (level[e.to] != level[v] + 1) {
                continue;
            }

            if (e.cap - e.flow <= 0) {
                continue;
            }

            ll tr = dfs(
                e.to,
                min(pushed, e.cap - e.flow)
            );

            if (tr == 0) {
                continue;
            }

            e.flow += tr;

            // Reverse edge
            edges[id ^ 1].flow -= tr;

            return tr;
        }

        return 0;
    }

    // Returns maximum flow
    ll max_flow() {
        ll totalFlow = 0;

        while (bfs()) {
            fill(ptr.begin(), ptr.end(), 0);

            while (true) {
```

# CÓDIGO · dinic

```
                ll pushed = dfs(source, INF);

                if (pushed == 0) {
                    break;
                }

                totalFlow += pushed;
            }
        }

        return totalFlow;
    }
};

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    /*
        BASIC USAGE
        -----------

        Suppose we have nodes:

            0, 1, 2, 3

        plus:

            source = 4
            sink   = 5

        Therefore:

            totalNodes = 6
    */

    int totalNodes = 6;
    int source = 4;
    int sink = 5;

    Dinic dinic(totalNodes, source, sink);

    /*
        Add directed edges:

        add_edge(from, to, capacity)
    */

    dinic.add_edge(source, 0, 3);
    dinic.add_edge(source, 1, 2);

    dinic.add_edge(0, 2, 2);
    dinic.add_edge(0, 3, 1);
    dinic.add_edge(1, 2, 1);
    dinic.add_edge(1, 3, 2);

    dinic.add_edge(2, sink, 2);
    dinic.add_edge(3, sink, 3);

    /*
        Run Dinic.
    */

    ll flow = dinic.max_flow();

    cout << "Maximum flow: " << flow << '\n';


    /*
        ----------------------------------------
        HOW TO INSPECT THE FLOW OF EVERY EDGE
        ----------------------------------------
```

# CÓDIGO · dinic

```
211
212         Every add_edge() adds TWO edges:
213
214             edges[id]     = original
215             edges[id ^ 1] = residual
216
217         Therefore, original edges are:
218
219             0, 2, 4, 6, ...
220
221         You can recover which edges were used by
222         checking:
223
224             e.flow > 0
225     */
226
227     for (int id = 0; id < (int)dinic.edges.size(); id += 2) {
228         FlowEdge& e = dinic.edges[id];
229
230         cout
231             << e.from
232             << " -> "
233             << e.to
234             << "   flow = "
235             << e.flow
236             << "/"
237             << e.cap
238             << '\n';
239     }
240
241
242     /*
243         ---------------------------------------
244         BIPARTITE MATCHING
245         ---------------------------------------
246
247         If you have:
248
249             Left side:
250                 0 ... L - 1
251
252             Right side:
253                 L ... L + R - 1
254
255             source = L + R
256             sink   = L + R + 1
257
258         then:
259
260             Dinic dinic(L + R + 2, source, sink);
261
262         source -> LEFT:
263             capacity 1
264
265         LEFT -> RIGHT:
266             capacity 1
267
268         RIGHT -> sink:
269             capacity 1
270
271
272         Example:
273
274             for (int i = 0; i < L; ++i) {
275                 dinic.add_edge(source, i, 1);
276             }
277
278             for (int j = 0; j < R; ++j) {
279                 dinic.add_edge(L + j, sink, 1);
280             }
281
282             if (left_i can match right_j) {
```

**# CÓDIGO · dinic**

```
                dinic.add_edge(i, L + j, 1);
            }

            ll matching = dinic.max_flow();


        ---------------------------------------
        RECOVER MATCHES
        ---------------------------------------

        for (int id = 0;
             id < (int)dinic.edges.size();
             id += 2) {

            FlowEdge& e = dinic.edges[id];

            bool fromLeft =
                e.from >= 0 &&
                e.from < L;

            bool toRight =
                e.to >= L &&
                e.to < L + R;

            if (fromLeft && toRight && e.flow == 1) {

                int left = e.from;
                int right = e.to - L;

                cout << left << " " << right << '\n';
            }
        }


        ---------------------------------------
        B-MATCHING
        ---------------------------------------

        If every node can be matched K times,
        just change the capacities:

            source -> left    capacity K
            right  -> sink    capacity K

        while:

            left -> right     capacity 1

        Example for degree <= 2:

            dinic.add_edge(source, left, 2);
            dinic.add_edge(right, sink, 2);

        The reconstruction is exactly the same:
        left -> right edges with flow == 1.
    */

    return 0;
}
```

# edmonds_karp

**GRAPH / FLOWS · C++ · 61 LÍNEAS**

**# CÓDIGO · edmonds_karp**

```cpp
template <int N>
class Graph {
public:
    Graph(int src, int snk)
        : source{src}, sink{snk} {
            q.reserve(N);
        }

    void add(int u, int v, long long w) {
        cap[u][v] += w;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    long long max_flow() {
        long long ans{};

        while (true) {
            auto flow {bfs()};
            if (flow == 0) break;
            ans += flow;

            int cur {sink};
            while (cur != source) {
                int prev {p[cur]};
                cap[prev][cur] -= flow;
                cap[cur][prev] += flow;
                cur = prev;
            }
        }
        return ans;
    }

private:
    int source{}, sink{};
    std::array< std::vector<int>, N> adj{};
    std::array< std::array<long long, N>, N> cap{};
    std::array<int, N> p{};
    std::vector< std::pair<int, long long>> q{};

    long long bfs() {
        std::ranges::fill(p, -1);
        p[source] = -2;

        q.clear();
        q.emplace_back(source, INF);

        for (int j{}; j < std::ssize(q); ++j) {
            auto [node, flow] {q[j]};
            for (auto u : adj[node]) {
                if (p[u] != -1 || !cap[node][u]) continue;
                p[u] = node;

                auto x { std::min(flow, cap[node][u]) };
                if (u == sink) return x;
                q.emplace_back(u, x);
            }
        }
        return 0;
    }
};
```

# min_cost_max_flow

**GRAPH / FLOWS · C++ · 84 LÍNEAS**

CÓDIGO · min_cost_max_flow

```cpp
const long long inf = 1e18+7;

struct FlowGraph {

    struct Edge {
        long long to, flow, cap, cost;
        Edge *res;

        Edge () : to(0), flow(0), cap(0), cost(0), res(0) {}
        Edge (long long to, long long flow, long long cap, long long cost) : to(to), flow(flow), cap(cap),
            cost(cost), res(0) {}

        void addFlow (long long f) {
            flow += f;
            res->flow -= f;
        }
    };

     long long n;
    vector<vector<Edge*>> adj;
    vector<long long> dis, pos;


    FlowGraph (int n) : n(n), adj(n), dis(n), pos(n) {}

    void add (int u, int v, long long cap, long long cost) {
        Edge *x = new Edge(v, 0, cap, cost);
        Edge *y = new Edge(u, cap, cap, -cost);
        x->res = y;
        y->res = x;
        adj[u].push_back(x);
        adj[v].push_back(y);
    }

    pair<long long, long long> mcmf(int s, int t, long long tope) {
        vector<bool> inq(n);
        vector<long long> dis(n), cap(n);
        vector<Edge*> par(n);
        long long maxFlow = 0, minCost = 0;

        while (maxFlow < tope) { //  compute MCF: maxflow < tope, compute MCMF maxflow < inf
            fill(dis.begin(), dis.end(), inf);
            fill(par.begin(), par.end(), nullptr);
            fill(cap.begin(), cap.end(), 0);
            dis[s] = 0;
            cap[s] = inf;
            queue<int> q;
            q.push(s);

            while ((int)q.size()) {
                int u = q.front();
                q.pop();
                inq[u] = 0;

                for (Edge *v : adj[u]) {
                    if (v->cap > v->flow && dis[v->to] > dis[u] + v->cost) {
                        dis[v->to] = dis[u] + v->cost;
                        par[v->to] = v;
                        cap[v->to] = min(cap[u], v->cap - v->flow);

                        if (!inq[v->to]) {
                            q.push(v->to);
                            inq[v->to] = 1;
                        }
                    }
                }
            }
```

# CÓDIGO · min_cost_max_flow

```
67
68              if (!par[t]) break;
69
70              maxFlow += cap[t];
71              minCost += cap[t] * dis[t];
72              for (int u = t; u != s; u = par[u]->res->to)
73                  par[u]->addFlow(cap[t]);
74          }
75
76          return {maxFlow, minCost};
77      }
78  };
79
80  void doit(){
81      // define src and sink.
82      // edges src to node, and node to sink have cost 0.
83      // to compute flow matches (e.g assignment problems), run dfs over the flow graph, keep the path and
            substract one unit of flow every time.
84  }
```

SECCIÓN 06

# Graph / Tree

2 algoritmos

# diameter

**GRAPH / TREE · C++ · 32 LÍNEAS**

CÓDIGO · diameter

```cpp
const int INF {1'000'000'000};
std::vector<int> depth(n, INF);

auto diameter = [&](int start) -> int {
    std::vector<int> q{start};
    depth[start] = 0;

    for (int i{}; i < std::ssize(q); ++i) {
        auto node {q[i]};
        for (auto u : adj[node]) {
            if (depth[node] > depth[u]) continue;
            depth[u] = depth[node] + 1;
            q.push_back(u);
        }
    }

    int ans {};
    auto ldp = [&](auto& dfs, int node, int parent) -> int {
        int max{};

        for (auto u : adj[node]) {
            if (parent == u) continue;

            int x = dfs(dfs, u, node);
            ans = std::max(ans, max + x + 1);
            max = std::max(max, x + 1);
        }
        return max;
    };
    ldp(ldp, start, -1);
    return ans;
};
```

# lowest_common_ancestor

**GRAPH / TREE · C++ · 46 LÍNEAS**

**CÓDIGO · lowest_common_ancestor**

```cpp
const int nax {200'005}, LOG {20}, INF {1'000'000'000};
static std::array< std::array<int, LOG>, nax> up{};
std::vector<int> depth(n, INF);

std::vector<int> q{0};
q.reserve(n);
depth[0] = 0;

for (int j{}; j < std::ssize(q); ++j) {
    auto node {q[j]};
    for (auto u : adj[node]) {
        if (depth[node] > depth[u]) continue;
        up[u][0] = node;
        depth[u] = depth[node] + 1;
        q.push_back(u);
    }
}

for (int j{1}; j < LOG; ++j) {
    for (int i{}; i < n; ++i) {
        up[i][j] = up[up[i][j - 1]][j - 1];
    }
}

auto walk  = [&](int node, int k) -> int {
    if (k > depth[node]) return -1;
    for (int i{}; i < LOG; ++i) {
        if (k & (1 << i)) {
            node = up[node][i];
        }
    }
    return node;
};

auto lca = [&](int x, int y) -> int {
    if (depth[x] < depth[y]) std::swap(x, y);
    x = walk(x, depth[x] - depth[y]);
    if (x == y) return x;
    for (int i{LOG - 1}; i >= 0; --i) {
        if (up[x][i] != up[y][i]) {
            x = up[x][i];
            y = up[y][i];
        }
    }
    return up[x][0];
};
```

SECCIÓN 07

# Math

7 algoritmos

# binary_exponentiation

**MATH · C++ · 13 LÍNEAS**

**CÓDIGO · binary_exponentiation**

```cpp
const int mod = 1e9 + 7;

auto power (long long a, long long exp) {
    long long r{1};
    while (exp) {
        if (exp % 2) {
            r = (a * r) % mod;
        }
        a = (a * a) % mod;
        exp /= 2;
    }
    return r;
}
```

# combinatorics

**MATH · C++ · 34 LÍNEAS**

**# CÓDIGO · combinatorics**

```cpp
const int nax {67};
auto pascal = []() {
    std::array< std::array<long long, nax>, nax> p{};
    p[0][0] = 1;
    for (int i{1}; i < nax; ++i) {
        p[i][0] = 1;
        for (int j{1}; j < nax; ++j) {
            p[i][j] += p[i - 1][j] + p[i - 1][j - 1];
        }
    }
    return p;
}();

const int nax {100'000};

std::vector<long long> invFacto(nax, 1);
const auto facto = []() {
    std::array<long long, nax> f;
    f[0] = 1;
    for (int i{1}; i < nax; ++i) {
        f[i] = multi(f[i - 1], i);
    }
    invFacto[nax - 1] = divide(1, facto[nax - 1]);
    for (int i{nax - 1}; i > 0; --i) {
        invFacto[i - 1] = multi(invFacto[i], i);
    }
    return f;
}();

long long nCk(int n, int k) {
    if (k < 0 || n < k) return 0;
    return pascal[n][k];
    return multi(facto[n], multi(invFacto[k], invFacto[n - k]));
}
```

# fft

MATH · C++ · 141 LÍNEAS

CÓDIGO · fft

```cpp
typedef pair<long long,long long> ii;

const long long MAXN=1<<20; //watch out with RTEs (increase MAXN):
typedef vector<long long> poly;
struct CD {
    double r,i;
    CD(double r=0, double i=0):r(r),i(i){}
    double real()const{return r;}
    void operator/=(const int c){r/=c, i/=c;}
};
CD operator*(const CD& a, const CD& b){
    return CD(a.r*b.r-a.i*b.i,a.r*b.i+a.i*b.r);}
CD operator+(const CD& a, const CD& b){return CD(a.r+b.r,a.i+b.i);}
CD operator-(const CD& a, const CD& b){return CD(a.r-b.r,a.i-b.i);}
const double pi=acos(-1.0);

CD cp1[MAXN+9],cp2[MAXN+9];
int R[MAXN+9];
void dft(CD* a, int n, bool inv){
    for(int i = 0; i < n; i++) if(R[i]<i)swap(a[R[i]],a[i]);
    for(int m=2;m<=n;m*=2){
        double z=2*pi/m*(inv?-1:1);
        CD wi=CD(cos(z),sin(z));
        for(int j=0;j<n;j+=m){
            CD w(1);
            for(int k=j,k2=j+m/2;k2<j+m;k++,k2++){
                CD u=a[k];CD v=a[k2]*w;a[k]=u+v;a[k2]=u-v;w=w*wi;
            }
        }
    }
    if(inv) for(int i = 0; i < n; i++) a[i]/=n;
}
poly multiply(poly& p1, poly& p2){
    int n=p1.size()+p2.size()+1;
    int m=1,cnt=0;
    while(m<=n)m+=m,cnt++;
    for(int i = 0; i < m; i++){R[i]=0;for(int j = 0; j < cnt; j++) R[i]=(R[i]<<1)|((i>>j)&1);}
    for(int i = 0; i < m; i++) cp1[i]=0,cp2[i]=0;
    for(int i = 0; i < (int)p1.size(); i++) cp1[i]=p1[i];
    for(int i = 0; i < (int)p2.size(); i++) cp2[i]=p2[i];
    dft(cp1,m,false);dft(cp2,m,false);
    for(int i = 0; i < m; i++) cp1[i]=cp1[i]*cp2[i];
    dft(cp1,m,true);
    poly res;
    n-=2;
    for(int i = 0; i < n; i++) res.push_back((long long)floor(cp1[i].real()+0.5));
    return res;
}

void getBigNumMulti(vector <long long> &c){ //Big numbers multiplication.
    vector <char> r;
    while(!c.empty()&&!c.back()) c.pop_back(); //quitar todos los 0 extras.
    if(c.empty()){
        cout<<0<<'\n';
        return;
    }
    long long x=0;
    //Normalizar los coeficientes para representarlos como digitos.
    for(int i = 0; i < (int)c.size(); i++){
        x+=c[i];
        r.push_back((char)(x%10) + '0');
        x/=10;
    }
    while(x){ //carry que sobra.
        r.push_back((char)(x%10) + '0');
        x/=10;
```

# CÓDIGO · fft

```
    }
    reverse(r.begin(), r.end());
    for(int i = 0; i < (int)r.size(); i++) cout<<r[i];
    cout<<'\n';
}

void stringMatchShift(){ //All possible scalar products with strings.
    string s;
    cin>>s;
    int n = s.size();
    vector <long long> a1(n,0),a2(2*n,0),b1(n,0),b2(2*n,0),c1(n,0),c2(2*n,0);
    vector <long long> ra,rb,rc;

    //Create binary polynomial for each letter.
    for(int i = 0; i < n; i++){
        if (s[i] == 'a') a1[i] = 1;
        else if (s[i] == 'b') b1[i] = 1;
        else c1[i] = 1;
    }
    //Make the dup for each letter to multiply:
    for(int i = 0; i < n; i++){
        a2[i] = a2[i+n] = a1[i];
        b2[i] = b2[i+n] = b1[i];
        c2[i] = c2[i+n] = c1[i];
    }
    //Append the rest of the zeros (Step 1):
    for(int i = 0; i < n; i++){
        a1.push_back(0), b1.push_back(0), c1.push_back(0);
    }
    //Reverse the arrays (Step 2):
    reverse(a1.begin(), a1.end());
    reverse(b1.begin(), b1.end());
    reverse(c1.begin(), c1.end());
    //Multiply the polynomials:
    ra = multiply(a1,a2);
    rb = multiply(b1,b2);
    rc = multiply(c1,c2);
    int shif = 1;
    //Left shift match (Step 1, then Step 2):
    for(int i = (2*n)-2; i>=n; i--){
        cout<<"L-shift: "<<shif<<" "<<ra[i] + rb[i] + rc[i]<<'\n';
        shif++;
    }
    //Right shift match (Step 2, then Step 1):
    shif = 1;
    for(int i = n-2; i>=0; i--){
        cout<<"R-shift: "<<shif<<" "<<ra[i] + rb[i] + rc[i]<<'\n';
        shif++;
    }
}

//String matching with wildcards ('?')
vector<long long> string_matching(string &s, string &t) {
    int n = s.size(), m = t.size();
    vector<long long> s1(n), s2(n), s3(n);
    //assign any non zero number for non '?'s
    for(int i = 0; i < n; i++) s1[i] = s[i] == '?' ? 0 : s[i] - 'a' + 1;
    for(int i = 0; i < n; i++) s2[i] = s1[i] * s1[i];
    for(int i = 0; i < n; i++) s3[i] = s1[i] * s2[i];
    vector<long long> t1(m), t2(m), t3(m);
    for(int i = 0; i < m; i++) t1[i] = t[i] == '?' ? 0 : t[i] - 'a' + 1;
    for(int i = 0; i < m; i++) t2[i] = t1[i] * t1[i];
    for(int i = 0; i < m; i++) t3[i] = t1[i] * t2[i];
    reverse(t1.begin(), t1.end());
    reverse(t2.begin(), t2.end());
    reverse(t3.begin(), t3.end());
    vector<long long> s1t3 = multiply(s1, t3);
    vector<long long> s2t2 = multiply(s2, t2);
    vector<long long> s3t1 = multiply(s3, t1);
    vector<long long> res(n);
    for(int i = 0; i < n; i++) res[i] = s1t3[i] - s2t2[i] * 2 + s3t1[i];
    vector<long long> oc;
```

**# CÓDIGO · fft**

```
    for(int i = m - 1; i < n; i++) if(res[i] == 0) oc.push_back(i - m + 1);
    return oc;
}
```

# matrix_expo

**MATH · C++ · 30 LÍNEAS**

CÓDIGO · matrix_expo

```cpp
const long long mod = 1e9+7;
long long tc,n,m,k;

vector<vector<long long>> mul(vector<vector<long long>> a, vector<vector<long long>> b) {
    vector<vector<long long>> c((int)a.size(), vector<long long>((int)b[0].size()));
    for (int i = 0; i < (int)a.size(); i++)
        for (int j = 0; j < (int)b.size(); j++)
            for ( int k = 0; k < (int)a.size(); k++)
                (c[i][j] += a[i][k]*b[k][j]%mod)%=mod; //for amount of paths.
                //c[i][j] = min(c[i][j], a[i][k] + b[k][j]); //for shortest path.
    return c;
}

vector<vector<long long>> exp( vector<vector<long long>> x, int y) { // matrix and desired power.
    vector<vector<long long>> r((int)x.size(), vector<long long>((int)x.size(),0ll)); //0ll: amount of
        paths. INF: shortest path
    for ( int i = 0; i < (int)x.size(); i++) r[i][i] = 1; //1ll: amount of paths. 0ll: shortest path.
    while (y>0){
        if (y&1) {
            r = mul(r,x);
        }
        y=y>>1;
        x = mul(x,x);
    }
    return r;
}

void doit(){
    // build adjacency (or costs) matrix of size(n*n).
    // after exponentiating mat[i][j] denotes the path from i to j.
}
```

# modular_arithmetic

**MATH · C++ · 20 LÍNEAS**

**# CÓDIGO · modular_arithmetic**

```cpp
const int mod = 1e9 + 7;
long long power(long long a, long long exp);

auto toMod(long long x) {
    return ((x % mod) + mod) % mod;
}
auto add(long long a, long long b) {
    auto ans {a + b};
    return (ans >= mod) ? ans - mod : ans;
}
auto sub(long long a, long long b) {
    auto ans {a - b};
    return (ans < 0) ? ans + mod : ans;
}
auto multi(long long a, long long b) {
    return (a * b) % mod;
}
auto divide(long long a, long long b) {
    return multi(a, power(b, mod - 2));
}
```

# precision

**MATH · C++ · 17 LÍNEAS**

**CÓDIGO · precision**

```cpp
constexpr double eps {1e-9};

bool equal(double a, double b) {
    if (std::abs(a - b) <= eps) return true;
    return std::abs(a - b) <= std::max(std::abs(a), std::abs(b)) * eps;
}
bool less(double a, double b) {
    return a < (b - eps);
}
bool lessEqual(double a, double b) {
    return a < (b + eps);
}
long long divCeil(long long x, long long y) {
    bool neg {(x < 0) != (y < 0)};
    if (neg) return x / y;
    return (x / y) + (x % y != 0);
}
```

# prime_theory

**MATH · C++ · 64 LÍNEAS**

CÓDIGO · prime_theory

```cpp
std::vector<long long> primes{};
const int nax {100'000};

const auto gc = []() {
    std::bitset<nax + 1> c{};
    c[0] = c[1] = true;
    primes.reserve((nax / std::log(nax)) * 1.2);

    for (int m{4}; m <= nax; m += 2) c[m] = true;
    primes.push_back(2);
    for (long long p{3}; p <= nax; p += 2) {
        if (c[p]) continue;
        primes.push_back(p);
        for (long long m{p * p}; m <= nax; m += p + p) {
            c[m] = true;
        }
    }
    return c;
}();
bool isPrime(long long m) {
    if (m <= nax) return !gc[m];
    for (auto p : primes) {
        if (p * p > m) break;
        if (m % p == 0) return false;
    }
    return true;
}
auto primeFactors(long long m) {
    std::vector<std::pair<long long,int>> f{};
    for (auto p : primes) {
        if (p * p > m) break;
        int cont{};
        while (m % p == 0) {
            ++cont;
            m /= p;
        }
        if (cont) f.emplace_back(p, cont);
    }
    if (m != 1) f.emplace_back(m, 1);
    return f;
}
auto eulerPhiSieve = []() {
    std::array <long long, nax + 1> a{};
    std::iota(a.begin(), a.end(), 0);
    for (int i{2}; i <= nax; ++i) {
        if (a[i] == i) {
            for (int j{i}; j <= nax; j += i) {
                a[j] = (a[j] / i) * (i - 1);
            }
        }
    }
    return a;
}();
auto eulerPhi(long long n) {
    if (n <= nax) return eulerPhiSieve[n];
    long long ans{n};
    for (auto p : primes) {
        if (p * p > n) break;
        if (n % p == 0) ans -= ans / p;
        while (n % p == 0) n /= p;
    }
    if (n != 1) ans -= ans / n;
    return ans;
}
```

SECCIÓN 08

# Misc

2 algoritmos

# jose

**MISC · C++ · 22 LÍNEAS**

CÓDIGO · jose

```cpp
int J(int n, int k) {
    if (n == 1) return 0; // 1 (1-index)
    return (J(n - 1, k) + k) % n;
}

int josephus(int n, int k) {
    int res = 0;
    for (int i = 1; i <= n; ++i){
        res = (res + k) % i;
    }
    return res; // + 1 (1-index)
}

std::vector<int> ans{};
int pos{1};
// 1-index
for (int i{}; i < n; ++i) {
    int r {n - i};
    pos = ((pos + k - 1) % r) + 1;
    ans.push_back(jose.find(pos));
    jose.kill(ans.back());
}
```

# random

**MISC · C/C++ Header · 20 LÍNEAS**

**# CÓDIGO · random**

```cpp
#ifndef RANDOM_H
#define RANDOM_H

#include <chrono>
#include <random>

namespace Random {
    inline std::mt19937_64 mt {
        static_cast<std::seed_seq::result_type> (
            std::chrono::steady_clock::now().time_since_epoch().count()
        )
    };

    template<typename T>
    T get(T min, T max) {
        return std::uniform_int_distribution<T>{min, max}(mt);
    }
}

#endif
```

SECCIÓN 09

# String

6 algoritmos

# aho_corasick

**STRING · C++ · 48 LÍNEAS**

**CÓDIGO · aho_corasick**

```cpp
const int MAXN = 1e6+10;
map<char, int> to[MAXN]; // if TLE change to array
string a[MAXN];
int lnk[MAXN],sz=1;
long long que[MAXN],endlink[MAXN];
vector<long long> leaf[MAXN],ans[MAXN];
void add_str(string s, int id) {
    int v = 0;
    for(char c: s) {
        if(!to[v].count(c)) to[v][c] = sz++;
        v = to[v][c];
    }
    leaf[v].push_back(id); //Node in automata where a word ends
}
void push_links() {
    queue<int> q({0});
    while(!q.empty()) {
        int v = q.front(); q.pop();
        for(auto [c,u]: to[v]){
            int j = lnk[v];
            while(j && !to[j].count(c)) j=lnk[j];
            if(to[j].count(c) && to[j][c]!=u) lnk[u] = to[j][c];
            endlink[u] = leaf[lnk[u]].size()>0?lnk[u]:endlink[lnk[u]];
            q.push(u);
        }
    }
}
void walk(string s){ //KMP with multiple target patterns
    int v=0;
    for(int i = 0; i < (int)s.size(); i++){
        char c=s[i];
        while(v && !to[v].count(c)) v=lnk[v];
        if(to[v].count(c)) v=to[v][c];
        for(int u=v;u;u=endlink[u]) for(int x: leaf[u]){
            ans[x].push_back(i); //pushing the index of the main string where a pattern ends
        }
    }
}
void doit(){
    string s;
    cin>>s; //main string.
    for(int i = 0; i < n; i++){
        cin>>a[i];
        add_str(a[i],i); //add target strings.
    }
    push_links();
    walk(s);
}
```

# hashing

STRING · C++ · 66 LÍNEAS

CÓDIGO · hashing

```cpp
#include <bits/stdc++.h>
using namespace std;

// To hash a multiset: Sum of elements of: (B^num) * freq[num]
const long long MOD = 1000000009LL; // prime modulo (alternative: 10000000000000061 or 10000000000061)
const long long BASE = 131LL; // any number > alphabet size and coprime to MOD.

string s, t;
vector<long long> pws, h;
long long hashT = 0, ans = 0;

long long convertChar(char c) {
    return static_cast<long long>(c - 'a' + 1);
}

bool sameHash(int l1, int len1, int l2, int len2) {
    int r1 = l1 + len1;
    int r2 = l2 + len2;
    long long h1 = (h[r1] - h[l1] * pws[len1] % MOD + MOD) % MOD;
    long long h2 = (h[r2] - h[l2] * pws[len2] % MOD + MOD) % MOD;
    return h1 == h2;
}

void precalc() {
    int lenS = static_cast<int>(s.size());
    int lenT = static_cast<int>(t.size());

    pws.assign(lenS + 1, 1);
    h.assign(lenS + 1, 0);

    for (int i = 1; i <= lenS; ++i) {
        pws[i] = (pws[i - 1] * BASE) % MOD;
    }

    for (int i = 0; i < lenS; ++i) {
        h[i + 1] = (h[i] * BASE + convertChar(s[i])) % MOD;
    }

    for (int i = 0; i < lenT; ++i) {
        hashT = (hashT * BASE + convertChar(t[i])) % MOD;
    }
}

void doit() {
    int lenS = static_cast<int>(s.size());
    int lenT = static_cast<int>(t.size());

    precalc();

    for (int i = lenT; i <= lenS; ++i) {
        long long curHash = (h[i] - h[i - lenT] * pws[lenT] % MOD + MOD) % MOD;
        if (curHash == hashT) {
            ++ans;
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> s >> t;
    doit();
    cout << ans << '\n';
    return 0;
}
```

# kmp

**STRING · C++ · 40 LÍNEAS**

CÓDIGO · kmp

```cpp
vector<long long> prefix_function(string s){
    long long n= (int)s.size();
    vector<long long> pi(n);

    for(int i = 1; i<n; i++){
        long long j=pi[i-1];

        while(j>0 && s[i]!=s[j])
            j=pi[j-1];

        if(s[i]==s[j])
            j++;

        pi[i]=j;
    }

    return pi;
}

vector<long long> kmp(string s,string pat){
    vector<long long> pi=prefix_function(pat);
    vector<long long> ans;

    long long j=0;

    for(int i = 0; i< (int)s.size(); i++){
        while(j>0 && s[i]!=pat[j])
            j=pi[j-1];

        if(s[i]==pat[j])
            j++;

        if(j==sz(pat)){
            ans.pb(i-sz(pat)+1);
            j=pi[j-1];
        }
    }

    return ans;
}
```

# suffix_array

**STRING · C++ · 308 LÍNEAS**

**# CÓDIGO · suffix_array**

```cpp
// Push back a sentinel {static_cast<char>(1)} to the string before SuffixArray
class SuffixArray {
public:
    SuffixArray (std::string_view a)
        : s{a}, n{static_cast<int>(std::ssize(a))},
        suffix(n), rank(n), lcp(n) {
            build();
            buildlcp();
        }
    int size() { return n; }
    const auto& get_lcp() { return lcp; }
    const auto& get_suffix() { return suffix; }
private:
    std::string_view s{};
    int n{};
    std::vector<int> suffix{}, rank{}, lcp{};

    void csort(int k) {
        int m{std::max(n, 300)};
        std::vector<int> f(m);
        for (int i{}; i < n; ++i) {
            ++f[(i + k < n) ? rank[i + k] : 0];
        }
        for (int i{1}; i < m; ++i) f[i] += f[i - 1];
        std::vector<int> b{suffix.rbegin(), suffix.rend()};
        for (auto u : b) {
            int r{(u + k < n) ? rank[u + k] : 0};
            suffix[--f[r]] = u;
        }
    }
    void build() {
        std::iota(suffix.begin(), suffix.end(), 0);
        for (int i{}; i < n; ++i) rank[i] = s[i];

        for (int k{1}; k < n; k *= 2) {
            csort(k);
            csort(0);
            std::vector<int> tr(n);
            int r{};
            tr[suffix[0]] = r;
            for (int i{1}; i < n; ++i) {
                int cur {suffix[i]};
                int prev {suffix[i - 1]};
                tr[cur] = (
                    (rank[cur] == rank[prev])
                &&  (rank[cur + k] == rank[prev + k])
                ) ? r : ++r;
            }
            std::swap(rank, tr);
            if (rank[suffix[n - 1]] == n - 1) break;
        }
    }
    void buildlcp() {
        std::vector<int> phi(n), plcp(n);
        phi[suffix[0]] = -1;
        for (int i{1}; i < n; ++i) phi[suffix[i]] = suffix[i - 1];
        for (int i{}, len{}; i < n; ++i) {
            if (phi[i] == -1) {
                plcp[i] = 0;
                continue;
            }
            while ((i + len < n) && (phi[i] + len < n)
                && (s[i + len] == s[phi[i] + len]) ) ++len;
            plcp[i] = len;
            len = std::max(len - 1, 0);
        }
```

# CÓDIGO · suffix_array

```
        for (int i{}; i < n; ++i) {
            lcp[i] = plcp[suffix[i]];
        }
    }
};

// Longest-Repeated-Substring O(N)
// Returns {length, start_index} of one occurrence (lexicographically smallest).
std::pair<int,int> lrs (SuffixArray& sa) {
    const auto& lcp {sa.get_lcp()};
    const auto& suffix {sa.get_suffix()};

    std::pair<int,int> ans{-1, -1};
    for (int i{}; i < sa.size(); ++i) {
        if (lcp[i] > ans.first) {
            ans = {lcp[i], suffix[i]};
        }
    }
    return ans;
}

// Longest-Repeated-Substring-Occurrence-Count O(N)
// Finds the longest substring that repeats, and how many times it occurs.
// Returns {length, occurrence_count} of that substring.
// If no substring repeats, returns {0, 1} (every substring occurs once).
std::pair<int,int> lrso(SuffixArray& sa) {
    const auto& lcp{sa.get_lcp()};
    int m{std::ssize(lcp)};

    int len{};
    for (int x : lcp) len = std::max(len, x);
    if (len == 0) return {0, 1};

    int ans{1}, run{};
    for (int i{}; i < m; ++i) {
        if (lcp[i] >= len) {
            ++run;
            ans = std::max(ans, run + 1);
        } else {
            run = 0;
        }
    }
    return {len, ans};
}

// Count-Distinct-Substrings O(N)
long long cds (SuffixArray& sa) {
    long long n{sa.size() - 1}, ans {(n * (n + 1)) / 2};
    for (auto u : sa.get_lcp()) ans -= u;
    return ans;
}

// Count-Distinct-Substrings-Repetead-At-Least-K O(N)
long long cdrk (SuffixArray& sa, int k) {
    const auto& lcp{sa.get_lcp()};
    int n {std::ssize(lcp)};
    if (k < 1 || n < k) return 0;
    if (k == 1) {
        return cds(sa);
    }

    std::deque<int> dq{};
    long long ans{};
    int x{};

    for (int i{1}; i < n; ++i) {
        while (!dq.empty() && lcp[dq.back()] >= lcp[i]) dq.pop_back();
        dq.push_back(i);
        if (dq.front() <= i - (k - 1)) dq.pop_front();
        int l{i - (k - 2)};
        if (l < 1) continue;

```

**# CÓDIGO · suffix_array**

```cpp
        ans += std::max(0, lcp[dq.front()] - x);
        x = lcp[dq.front()];
    }
    return ans;
}

std::string kth_distinct_substring(SuffixArray& sa, const std::string& s, long long k) {
    const auto& suf = sa.get_suffix();
    const auto& lcp = sa.get_lcp();
    const int n = std::ssize(s) - 1;

    for (int i{}; i < std::ssize(suf); ++i) {
        long long prev = (i == 0) ? 0 : lcp[i];
        long long m = (n - suf[i]) - prev;
        if (m <= 0) continue;
        if (k <= m) return s.substr(suf[i], prev + k);
        k -= m;
    }
    return "";
}

// Longest-Repetead-Substring-At-Least-K O(N)
// Returns {length, start_index} of one valid occurrence — specifically
// the occurrence with the largest/smallest starting index among all substrings
std::pair<int,int> lrsk(SuffixArray& a, int k) {
    int n{a.size()};
    int m {k - 1};
    if (m > n - 1) return {0, 0};
    if (k <= 1) return {n - 1, 0};

    const auto& lcp {a.get_lcp()};
    const auto& suffix {a.get_suffix()};
    std::deque<int> dq, dqpos{};

    int best{}, idx{-1};
    for (int i{}; i < n; ++i) {
        while (!dq.empty() && lcp[dq.back()] >= lcp[i]) dq.pop_back();
        dq.push_back(i);
        if (dq.front() <= i - m) dq.pop_front();

        while (!dqpos.empty() && suffix[dqpos.back()] <= suffix[i]) dqpos.pop_back();
        dqpos.push_back(i);
        if (dqpos.front() < i - m) dqpos.pop_front();

        if (i >= m) {
            int maxpos {suffix[dqpos.front()]};
            if (best < lcp[dq.front()]) {
                best = lcp[dq.front()];
                idx = maxpos;
            } else if (best == lcp[dq.front()] && best > 0) {
                idx = std::max(idx, maxpos);
            }
        }
    }
    if (best == 0) return {};
    return {best, idx};
}

// Longest-Repetead-Substring-By-Lenght O(N)
// Returns a vector of size n+1; index L (1..n) holds max occurrences of
// length-L substrings.
std::vector<int> lrbl(SuffixArray& sa) {
    int n {sa.size()};
    const auto& lcp {sa.get_lcp()};
    std::vector<int> left(n), right(n), stk{-1};
    stk.reserve(n + 1);

    for (int i{}; i < n; ++i) {
        while (stk.back() != -1 && lcp[stk.back()] >= lcp[i]) stk.pop_back();
        left[i] = stk.back();
        stk.push_back(i);
    }
```

# CÓDIGO · suffix_array

```cpp

    stk.clear();
    stk.push_back(n);
    for (int i{n - 1}; i >= 0; --i) {
        while (stk.back() != n && lcp[stk.back()] >= lcp[i]) stk.pop_back();
        right[i] = stk.back();
        stk.push_back(i);
    }

    std::vector<int> ans(n + 1, 1);
    for (int i{}; i < n; ++i) {
        int width = right[i] - left[i] - 1;
        ans[lcp[i]] = std::max(ans[lcp[i]], width + 1);
    }

    for (int i{n - 1}; i >= 1; --i) {
        ans[i] = std::max(ans[i], ans[i + 1]);
    }
    return ans;
}

// LCP(i, j): longest common prefix of suffixes starting at i, j.
// i == j ---> n - i (whole suffix, no RMQ).
// i != j ---> let lo,hi = minmax(rank[i], rank[j]); RMQ(lcp[lo+1..hi]).

// Helper function to concatend strings for lcks
// preprocess the suffix_belongs
// Return string, ready to SuffixArray
int offset {105};
std::vector<int> suffix_belongs{};
std::vector<int> concatenated_strings (std::vector<std::string>& s) {
    std::vector<int> a{};
    suffix_belongs.clear();
    int maxn{};
    for (auto& u : s) maxn += std::ssize(u) + 1;
    a.reserve(maxn);
    suffix_belongs.reserve(maxn);

    for (int i{}; i < std::ssize(s); ++i) {
        for (auto c : s[i]) {
            a.push_back(c + offset);
            suffix_belongs.push_back(i);
        }
        a.push_back(static_cast<char>(1 + i));
        suffix_belongs.push_back(-1);
    }
    return a;
}

// Longest-Common-Substring-At-Least-K-Different Strings
// Finding the longest common substring that appears in at least K
// different strings out of a given collection of strings.
// Return the max {length, vector<>{index}}
// Most of the time should work with integers and offset them to a wider range
// 1->(k + 1) sentinel char, A = (k + 1) -> Z = (k + 1 + 'A')
std::pair<int, std::vector<int>> lcsk (SuffixArray& sa, int k) {
    int n {sa.size()};
    const auto& lcp {sa.get_lcp()};
    const auto& suffix {sa.get_suffix()};

    std::unordered_map<int, int> f{};
    int distinct{};
    std::deque<int> dq{};


    auto addSuffix = [&](int i) {
        if (i < 0) return;
        if (f[i]++ == 0) ++distinct;
    };
    auto rmSuffix = [&](int i) {
        if (i < 0) return;
        if (--f[i] == 0) --distinct;
```

# CÓDIGO · suffix_array

```
    };

     std::pair<int, std::vector<int>> ans{};

    int l{};
    for (int i{}; i < n; ++i) {
        addSuffix(suffix_belongs[suffix[i]]);
        while ((!dq.empty()) && lcp[dq.back()] >= lcp[i]) dq.pop_back();
        if (i > l) dq.push_back(i);

        while (distinct >= k) {
            if (!dq.empty()) {
                int len {lcp[dq.front()]};
                if (len > ans.first) {
                    ans = {len, std::vector{suffix[l]}};
                } else if (len == ans.first && len > 0) {
                    ans.second.push_back(suffix[l]);
                }
            }
            rmSuffix(suffix_belongs[suffix[l]]);
            if ((!dq.empty()) && dq.front() == l + 1) dq.pop_front();
            ++l;
        }
    }
    return ans;
}
```

# Trie

**STRING · C++ · 64 LÍNEAS**

**CÓDIGO · Trie**

```cpp
struct Trie {
    struct Node {
        int nxt[26];
        bool end;

        Node() {
            fill(nxt, nxt + 26, -1);
            end = false;
        }
    };

    vector<Node> tr;

    Trie() {
        tr.emplace_back();
    }

    void insert(const string& s) {
        int u = 0;

        for (char c : s) {
            int x = c - 'a';

            if (tr[u].nxt[x] == -1) {
                tr[u].nxt[x] = (int)tr.size();
                tr.emplace_back();
            }

            u = tr[u].nxt[x];
        }

        tr[u].end = true;
    }

    bool find(const string& s) {
        int u = 0;

        for (char c : s) {
            int x = c - 'a';

            if (tr[u].nxt[x] == -1)
                return false;

            u = tr[u].nxt[x];
        }

        return tr[u].end;
    }

    bool prefix(const string& s) {
        int u = 0;

        for (char c : s) {
            int x = c - 'a';

            if (tr[u].nxt[x] == -1)
                return false;

            u = tr[u].nxt[x];
        }

        return true;
    }
};
```

# z_funtion

**STRING · C++ · 21 LÍNEAS**

CÓDIGO · z_funtion

```cpp
vector<long long> z_function(string s){
    ll n=(int)s.size();
    vector<long long> z(n);
    long long l=0,r=0;

    for(int i = 1; i<n ; i++){
        if(i<=r)
            z[i]=min(r-i+1,z[i-l]);

        while(i+z[i]<n && s[z[i]]==s[i+z[i]])
            z[i]++;

        if(i+z[i]-1>r){
            l=i;
            r=i+z[i]-1;
        }
    }

    return z;
}

```